

Web Application Hosting on AWS - Milestone

Deadline: 5/6/2026 at 11:59PM

Description: Design and implement a simple web application using any JavaScript stack of your choice. The application should perform CRUD (Create, Read, Update, Delete) operations on a dataset of any domain of interest of your choice. The application will be hosted on EC2 instances and will use different AWS services including DynamoDB for database storage, S3 for image uploads, and AWS SDK for interacting with AWS services.

Requirements:

- **Technology Stack:**
 - Choose any JavaScript stack for developing the web application. Examples include MEAN (MongoDB, Express.js, AngularJS, Node.js), MERN (MongoDB, Express.js, React, Node.js), or any other stack (you should use DynamoDB instead of Mongo).
- **CRUD Operations:**
 - Implement functionality to perform Create, Read, Update, and Delete operations on a dataset. The dataset can be related to any domain of interest you choose.
- **High Availability Architecture:**
 - Design the architecture for high availability by deploying the application across multiple EC2 instances in different Availability Zones within a region. Include **CloudFront** for faster delivery of your application.
- **AWS SDK Usage:**
 - Use AWS SDK for JavaScript to interact with the mentioned AWS services programmatically from your application (backend).
- **DynamoDB Integration:**
 - Use DynamoDB to store and manage the dataset. Design the table with at least three attributes.
- **S3 Image Upload:**
 - Implement functionality to upload images for newly created items in the application. Store the images in an S3 bucket. Ensure that the images are displayed or deleted when the corresponding item in the table is retrieved or

deleted. Also, allow for updating the image associated with an item, and ensure that both old and new versions are retained.

- **Link Images to Items:**
 - Associate the uploaded images stored in the S3 bucket with their corresponding items in the DynamoDB table.
- **Lambda Function:**
 - Create a Lambda function that should be triggered only upon the creation of added items. This function is responsible for resizing images uploaded to an S3 bucket. If needed, you may create an additional bucket for this purpose.

Deliverables:

- Detailed architecture diagram illustrating the high availability setup (*The detailed architecture must be drawn using AWS standard icons outlined in this link: <https://aws.amazon.com/architecture/icons/>, either through a PowerPoint presentation or one of the associated tools such as draw.io, Lucidchart..*)
- Source code of the web application (GitHub repo link)
- Presentation/demo highlighting the functionality and features of the web application
- DNS of your deployed app on AWS

Submission:

- Submit the [form](https://forms.gle/yo1D6cuD72VvWz4AA) (<https://forms.gle/yo1D6cuD72VvWz4AA>). Include the following: Link of the GitHub repo of your app, DNS of your ELB, the domain name of your CloudFront distribution, and Private IPs of your EC2s. Attach the architecture diagram you designed, and the demo.

Remember **NOT** to terminate any instances or used resources after your submission. Instead, simply stop them. Terminating any resources will result in a loss of progress or data, which will be considered as zero.

Evaluation Criteria:

- Adherence to project requirements and guidelines
- Functionality and usability of the web application
- Quality of architecture design and implementation
- Effective usage of AWS services and SDK
- Clarity and thoroughness of documentation and presentation
- Teams will be evaluated and graded both individually and as a group

****Note:** It is important to keep in mind the limitations of the AWS Free Tier to avoid exceeding budget constraints. ***Also, always review the pricing of each service before launching to ensure compatibility with free-tier usage limits.***